SCoPe

Trust-Region Latent Planning with JEPA World Models

How we built it: the whole project, step by step, in plain language

22AIE401 Reinforcement Learning
Aayush Shenday (AM.SC.U4AIE23004)
Rajashekar Kasarla (AM.SC.U4AIE23041)
Faculty: Dr. Geeta
Amrita School of Computing, Amritapuri

Contents

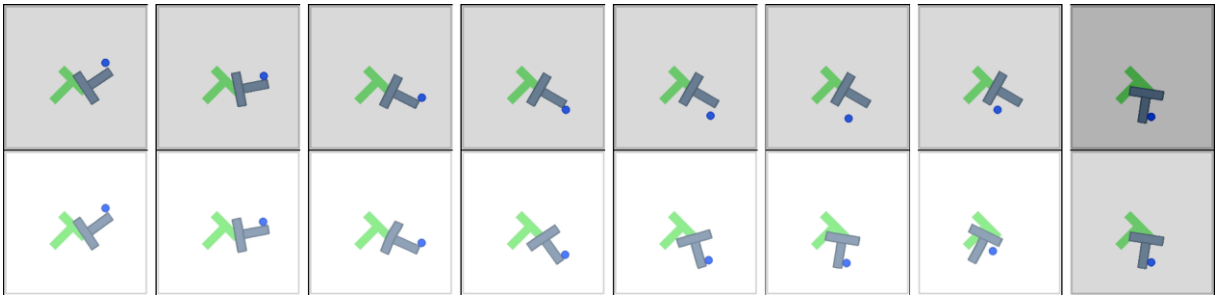
1. The project in one page 3
2. What is Reinforcement Learning? 3
3. The MDP of our project 4
4. RL concepts and methods we use 5
5. Are we making our own model? 7
6. What we used 8
7. Step-by-step implementation 8
8. How one planning round works 12
9. The numbers explained: rounds, iterations, plans, seeds 12
10. Results 13
11. What is new in our project 15
12. Limitations and future work 15
13. Glossary 16

1. The project in one page

A robot can decide what to do by **imagining** the future. It has a learned "imagination", called a **world model**, which predicts what will happen if it performs certain actions. The robot tries many possible action sequences in its imagination, picks the one whose imagined result is closest to the goal, and then performs it for real.

The problem: the imagination is sometimes wrong. The robot then confidently chooses a plan that looks perfect in its head but fails in reality. Today's planners never check whether the imagination can be trusted.

Our idea, SCoPe: before the robot acts, give every imagined step a **trust score** that says how likely it is to be wrong, and throw away plans that rely on untrustworthy steps. The robot then only chooses among plans it can trust.



The problem in one picture (Push-T task). Top row: what really happened. Bottom row: what the world model imagined for the same actions. In imagination the T-block reaches the goal; in reality it barely moves.

What we did, in four parts:

1. We took an existing, published world model and planner called **DINO-WM**, got it running on our laptop GPU, and checked that we get the same results as its authors.
2. We built a way to measure **how wrong** each imagined step really is, by running the same plans in the simulator and comparing.
3. We tested **five trust scores** to see which one best detects the wrong predictions.
4. We tested what happens when we **throw away** the plans with low trust.

What we found: we matched the published results (90%, 95% and 100% success on three tasks); one trust score, called **MC-dropout**, detects the wrong predictions much better than the alternatives; and throwing away plans only at the very last moment is not enough, so the trust check must be used throughout the planning.

2. What is Reinforcement Learning?

Reinforcement learning (RL) is a way for a computer agent to learn how to act. The agent is placed in an **environment**, looks at the situation, takes an **action**, sees what happens, and repeats. Over time it gets better at reaching its goal.

> **In simple words**
> Think of learning to ride a bicycle. Nobody gives you a list of correct muscle movements. You try something, you see whether you wobble or stay upright, and you adjust. RL works the same way: act, observe the result, improve.

2.1 The basic words of RL

RL word	Meaning in simple words	In our project
Agent	The one that decides and acts	The planner that controls the pusher, the dot or the ball
Environment	The world the agent acts in	A computer simulator of the task (Push-T, Wall or PointMaze)
State	Everything describing the current situation	A camera image of the scene plus the agent's position
Action	What the agent can do	A small movement of the agent
Transition	How the world changes after an action	The physics of the simulator
Reward / cost	A number saying how good or bad the result is	How far the (imagined) result is from the goal image
Episode	One complete attempt from start to end	One attempt to reach one goal from one starting position
Policy	The rule that decides the action	Our planner: "imagine, compare, pick the best"

2.2 Two families of RL

- **Model-free RL** (for example DQN, PPO, SAC): the agent learns directly by trial and error, which needs a huge number of attempts and a reward signal.
- **Model-based RL**: the agent first learns a **model of the world** (how things change after actions), and then uses that model to **plan** in its head before acting. It needs far fewer real attempts.

> **In simple words**
> **Our project is model-based RL.** The agent has a world model and plans with it. Our contribution is about making that planning safer: checking whether the world model's imagination can be trusted before acting on it.

3. The MDP of our project

In RL, a problem is described as a **Markov Decision Process (MDP)**. It is simply a precise way to say what the situations, the choices, the consequences and the goal are. "Markov" means that the current situation contains everything needed to decide what to do next; you do not need to remember the whole past.

Part of the MDP	What it is in our project
States (S)	A 224 × 224 camera image of the scene plus the agent's own position. The world model turns the image into a compact list of numbers, the "latent state", which is what it works with.
Actions (A)	A small movement of the agent (left/right and up/down). Actions are given in groups of five, so one planning step covers five small movements.

Part of the MDP	What it is in our project
Transitions (P)	What happens after an action: decided by the simulator's physics in reality, and predicted by the world model in imagination.
Reward / cost (R)	There is no hand-written reward. The planner uses a cost: how far the imagined final state is from the goal image. Smaller cost = better plan.
Goal	Given as a picture of the desired final situation (for example, the T-block lying on the target).
Horizon	The planner looks 5 planning steps (25 small movements) ahead. An episode lasts at most 3 rounds of planning.
Success	At the end, the simulator checks whether the goal has been reached (for Push-T: the block is close enough to the target in position and angle).

Our addition to the MDP (SCoPe): the planner must still find the plan with the lowest cost, but now with an extra condition: **every imagined step of the plan must be trusted**. A plan with even one untrusted step is not allowed. In maths this is written as "minimise the cost, subject to trust score ≤ threshold for every step", but the idea is just that sentence.

> **In simple words**
> Why is this a sequential decision problem and not just a prediction problem? Because an action's effect shows up several steps later (a push moves the block only after the pusher reaches it), and whether the goal was reached is only known at the end. The agent must plan a sequence, not a single move.

4. RL concepts and methods we use

Below is every RL idea that appears in the project, what it means, and where we use it.

4.1 Agent–environment loop

The basic RL cycle: observe the situation, choose an action, act, observe the new situation, repeat.

Where we use it: Every episode. The planner observes the image, plans, moves the agent in the simulator, observes again.

4.2 Model-based RL and the world model

Instead of learning only by trial and error, the agent has a model that predicts the next state from the current state and an action. This is the agent's "imagination".

Where we use it: We use the DINO-WM world model. It predicts how the scene will look (in compact form) after a sequence of actions.

4.3 Planning

Using the world model to try out many action sequences in imagination and choosing the best one before acting.

Where we use it: At every decision, the planner imagines hundreds of possible action sequences and picks the one that gets closest to the goal.

4.4 Cross-Entropy Method (CEM): the search

A simple way to search for a good plan. (1) Make 300 random plans. (2) Imagine each one and score it. (3) Keep the 30 best. (4) Make 300 new plans that look similar to those 30. (5) Repeat. The plans get better and better.

Where we use it: This is how the planner searches. On Push-T it repeats this 30 times; on Wall and PointMaze 10 times.

> **Example**
> Imagine throwing 300 darts at a board with your eyes closed. You open your eyes, find the 30 darts closest to the bullseye, and throw your next 300 darts around those. After a few rounds, almost all darts land near the bullseye. CEM does this with plans instead of darts.

4.5 Model Predictive Control (MPC): plan, act a little, plan again

Plans made in imagination are never perfect. So instead of executing the whole plan blindly, the agent executes only the first part, looks at the real result, and plans again from there. This is also called "receding horizon" planning.

Where we use it: After each plan, the agent performs the first 5 groups of actions, takes a new picture, and plans again. One plan-and-act cycle is called a **round**. We allow up to 3 rounds per episode.

4.6 Goal-conditioned control without a reward

The agent is given a goal (here, a picture of the goal) instead of a reward function. It tries to make the current situation look like the goal.

Where we use it: The planner's cost is the distance between the imagined final state and the goal picture.

4.7 Learning from recorded data (offline data)

The world model was trained on recordings of the agent moving around, not by live trial and error.

Where we use it: DINO-WM's authors trained it on such recordings. We use their trained model and their recordings. We do not train anything ourselves.

4.8 Model uncertainty and the trust region

A learned model is reliable in situations similar to what it has seen, and unreliable elsewhere. The set of situations where it is reliable is what we call the **trust region**.

Where we use it: SCoPe's whole idea: estimate the uncertainty of each imagined step (the trust score) and only act on plans that stay in the trust region.

4.9 MC-dropout: our best trust score

Neural networks are often trained with "dropout": parts of the network are randomly switched off so that it does not depend too much on any one part. If we keep dropout switched on while predicting, and predict the same thing four times, we get four slightly different answers. When the model knows the situation well, the four answers agree. When it does not, they disagree. The amount of disagreement is the trust score.

Where we use it: This is our best trust score. It needs no extra training.

Example
Ask four friends for directions. If all four say "second left", you can trust it. If they all say something different, nobody really knows. MC-dropout asks the model four times and checks whether the answers agree.

4.10 Exploration vs exploitation

A classic RL trade-off: try new things to learn (explore) or use what you know (exploit).

Where we use it: CEM explores by sampling many random plans and exploits by narrowing in on the best ones. The trust check adds caution: do not exploit predictions that are probably wrong.

5. Are we making our own model?

No. We do not build or train a world model. Training one like DINO-WM needs many days on large GPUs. We use the trained world model that the DINO-WM authors published, exactly as they released it. This is normal in research: it makes our results directly comparable with theirs.

Part	Who made it	What we did with it
Image encoder (DINOv2)	Meta AI, pretrained on millions of images	Used as is (frozen). It turns an image into a compact list of numbers.
World model (predictor)	DINO-WM authors (ICML 2025)	Used as is (frozen). It predicts the next compact state after an action.
Planner (CEM + MPC)	DINO-WM authors	Used, and made 3.8× faster without changing what it decides.
Datasets and the three tasks	DINO-WM authors and earlier work	Downloaded and used for the experiments.
Trust scores (5 of them)	**Us**	Designed and implemented.
Measuring the true error of imagined steps	**Us**	Designed and implemented (run each plan in the simulator and compare).
Trust gate (removing untrusted plans)	**Us**	Designed, implemented and tested.
Experiments, analysis, report	**Us**	All runs, numbers, figures, report and slides.

In simple words
"Frozen" means we never change the model. We only read its predictions. So the question we answer is not "how do we train a better model?" but "given a trained model, how can the planner tell when to trust it?"

6. What we used

6.1 The three tasks

Task	What the agent must do	Why it is useful
Push-T	A small round pusher must push a T-shaped block so that it lies exactly on a T-shaped target.	Hard: the block only moves when pushed, and it can rotate. Our main task.
Wall	A dot must move from one side of a wall to the other through a door.	Needs planning around an obstacle.
PointMaze	A ball must roll through a maze to a goal position.	Uses a realistic physics engine (MuJoCo).

6.2 Hardware and software

- **Computer:** a laptop with an NVIDIA RTX 5060 graphics card (8 GB of memory).
- **Operating system:** Windows 11 with WSL2 (a Linux system running inside Windows), because the DINO-WM code needs Linux.
- **Programming:** Python, PyTorch (the deep-learning library), CUDA (to use the graphics card).
- **Simulators:** pymunk (Push-T), MuJoCo (PointMaze), and a simple 2-D simulator (Wall).
- **Writing and sharing:** LaTeX (MiKTeX) for the report, PowerPoint for the slides, Git and GitHub for the code.

7. Step-by-step implementation

This is the order in which we built the project. Each step lists what we used, what we did and what came out of it.

Step 1: Set up the computer

What we used	• Windows 11 with WSL2 (Ubuntu Linux) • Conda (a tool that installs Python packages) • PyTorch with CUDA 12.8 (needed for the new RTX 5060 card)
What we did	• Installed Linux inside Windows, created a separate Python environment for the project, and installed PyTorch and the simulators. • Checked that PyTorch can see and use the graphics card.
Result	A working setup that can run DINO-WM.

Step 2: Download DINO-WM and its data

What we used	• The DINO-WM code from GitHub (fixed at one exact version so results are repeatable) • The authors' recorded datasets and trained models
What we did	• Downloaded the code, datasets and trained models for the three tasks. • Checked file sizes and checksums before unpacking, and kept large archives on a second drive to save space.
Result	All three tasks ready to run with the published trained models.

Step 3: Make it fit and run fast on our GPU

What we used	• PyTorch performance settings • A small benchmark script we wrote
What we did	• The original code did not fit in 8 GB of memory and took 26.2 seconds to imagine one batch of 300 plans. • We removed repeated work (for example, the starting picture was processed 300 times; now it is processed once), split the work into smaller pieces, and switched on faster maths settings on the GPU. • We compared old and new code on the same inputs and checked that the planner keeps exactly the same 30 best plans. One even faster setting changed the plans, so we did not use it.
Result	6.9 seconds per batch instead of 26.2: **3.8 times faster**, with identical decisions.

Step 4: Reproduce DINO-WM's results

What we used	• The planner (CEM + MPC) with the authors' settings • 50 start–goal pairs per seed, seeds 99, 100 and 101 • Up to 3 planning rounds per episode
What we did	• Ran the planner on Push-T (3 seeds), Wall (3 seeds) and PointMaze (1 seed). • Recorded after each round whether the goal was reached, plus pictures and videos of the runs. • Each Push-T seed took about 3 to 4.5 hours on our GPU.
Result	Success after replanning: Push-T 0.90, Wall 0.95, PointMaze 1.00 (published: 0.90, 0.96, 0.98). **We match the paper.**

Step 5: Design the trust scores

What we used	• The frozen world model itself • The recorded training data (for one of the scores)
What we did	• Designed five ways to score how trustworthy an imagined step is, none of which needs training: • MC-dropout: predict four times with dropout on and measure the disagreement. • Action perturbation: change the actions slightly and see how much the prediction changes. • Image perturbation: change the starting picture slightly and see how much the prediction changes. • Distance to training data: how far the imagined state is from anything seen during training. • Imagined cost: the planner's own idea of how good the plan is. • Added two simple baselines to compare against: a random score, and "distance moved" (how far the imagined state is from the start).
Result	Five trust scores and two baselines, all computed for every imagined step.

Step 6: Measure how wrong each imagined step really is

What we used	• The simulator, which can be reset to any exact starting state • 80 Push-T start–goal pairs over 3 seeds (40 + 20 + 20)
What we did	• For each start–goal pair, recorded 192 of the plans the planner looked at, from bad early plans to good late ones. • For each recorded plan: let the world model imagine it, then reset the simulator to the same start and actually perform the plan. • Took a picture after each step, turned it into the same compact form, and compared it with the imagined one. The difference is the **true error** of that imagined step. • Marked the 10% of steps with the largest error as **wrong** (separately for step 1, step 2, and so on, because later steps are naturally less accurate).
Result	80 × 192 × 5 = **76,800 imagined steps**, each with its true error known.

Step 7: Test which trust score works

What we used	• The 76,800 labelled steps from Step 6 • AUROC, a standard measure of how well a score separates wrong from right (explained below) • Bootstrap: repeating the calculation on random re-samples to get a 95% confidence range
What we did	• Computed every trust score for every imagined step. • Measured how well each score puts the wrong steps above the correct ones.
Result	**MC-dropout is best: AUROC 0.86** (range 0.84–0.88), better than the simple baseline (0.79) and random guessing (0.50), on each of the three seeds.

> **In simple words**
> **AUROC** answers one question: if you pick one wrong step and one correct step at random, how often does the score give the wrong one a higher (more suspicious) value? 0.5 means it is just guessing; 1.0 means it is always right. MC-dropout gets it right 86% of the time.

Step 8: Test removing untrusted plans (the gate)

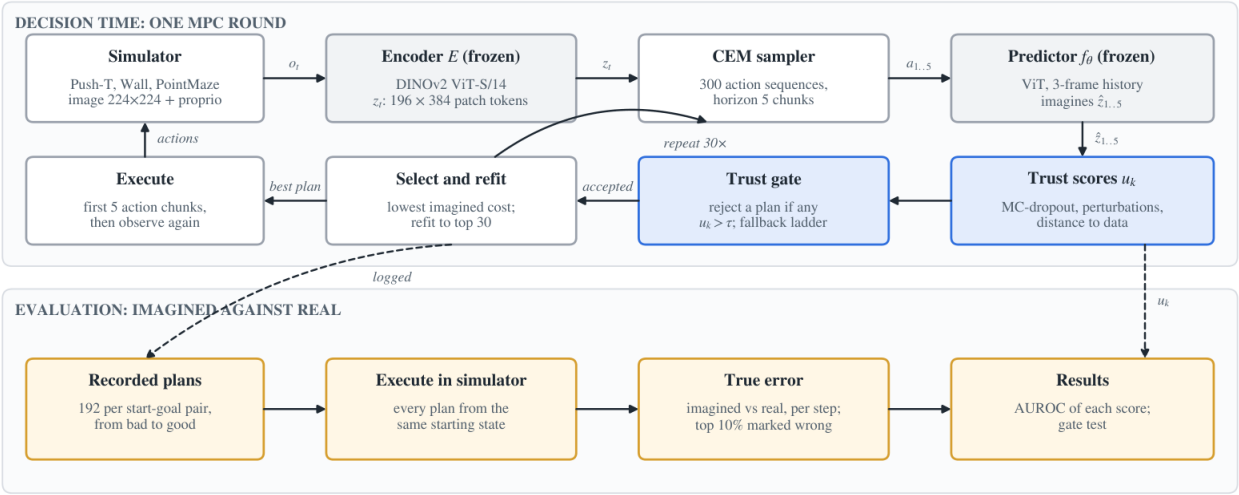
What we used	• The 192 recorded plans per start–goal pair • Each trust score, plus a random remover and an "oracle" that knows the true errors
What we did	• For each start–goal pair: removed the 30% least trusted plans, then picked the plan with the best imagined cost, then checked in the simulator whether that plan succeeds. • Compared with no removal, random removal, and the oracle.
Result	Success stays about the same: 71% with no removal, 70% with MC-dropout, and only 73% even with the oracle. The plan that the planner picks is rarely a badly predicted one, so removing plans **only at the last moment** changes little. The lesson: the trust check must guide the whole search (every CEM iteration and round), not just the final choice.

Step 9: Write up and share

What we used	• LaTeX (IEEE conference format), Python for tables and charts, PowerPoint, GitHub
What we did	• Generated every table and chart directly from the result files, so no number is typed by hand. • Wrote the report, built the slides, and published the code and results in three GitHub repositories grouped in one GitHub Project.
Result	Final report, presentation, and public repositories.

8. How one planning round works

This is what happens inside the agent every time it makes a decision.



SCoPe architecture. Top: one planning round. Grey: the frozen DINO-WM parts. Blue: the parts we add (trust scores and gate). Bottom: how we measured the true error.

1. **Look.** The simulator gives a picture of the scene. The encoder turns it into a compact state.
2. **Guess plans.** CEM makes 300 random action sequences, each 5 steps long.
3. **Imagine.** The world model predicts what happens for each of the 300 plans, step by step.
4. **Check trust (SCoPe).** Every imagined step gets a trust score.
5. **Remove untrusted plans (SCoPe).** Plans with an untrusted step are thrown away. If too few remain, the planner searches more widely, plans a shorter distance, or makes a safe move.
6. **Choose.** The plans that end closest to the goal are kept (the best 30).
7. **Improve.** CEM makes 300 new plans similar to the best ones and goes back to step 3. This repeats 30 times on Push-T.
8. **Act.** The agent performs the first part of the best plan in the simulator.
9. **Repeat.** It takes a new picture and starts a new round (up to 3 rounds).

9. The numbers explained: rounds, iterations, plans, seeds

Several different counts appear in the project. They are easy to mix up, so here they are side by side.

Word	What it means	How many
Plan (candidate)	One possible sequence of actions that the planner imagines	300 at a time
CEM iteration	One "imagine 300 plans, keep the best 30, make 300 new ones" cycle	30 per round on Push-T, 10 on Wall and PointMaze
Round (MPC round)	One "plan, then perform the first part for real, then look again" cycle	Up to 3 per episode
Episode (start–goal pair)	One complete attempt to reach one goal from one starting position	50 per seed

Word	What it means	How many
Seed	A number that decides which 50 start–goal pairs are picked	3 seeds (99, 100, 101); 1 for PointMaze
Recorded plans	Plans we saved to check against reality (Step 6)	192 per start–goal pair
Imagined steps checked	Recorded plans × 5 steps, compared with reality	76,800 in total

> **In simple words**
> **"300" is not the number of rounds.** It is the number of plans the planner imagines at once. A round is one plan-and-act cycle, and there are at most 3 of them per episode.

> **Example**
> One Push-T episode: up to 3 rounds × 30 CEM iterations × 300 plans = up to 27,000 imagined plans. One seed has 50 episodes, so up to 1.35 million imagined plans per seed. That is why one seed takes hours, and why making the code 3.8 times faster mattered.

What is a seed, in clear words?

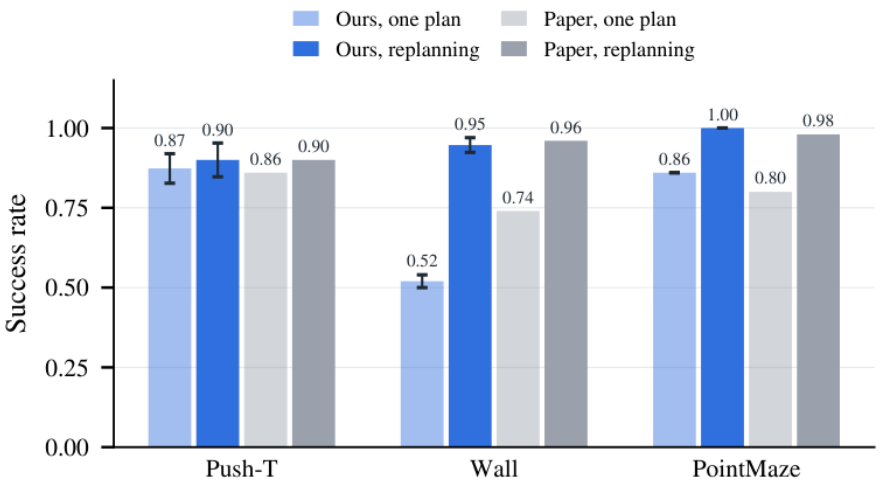
Computers use "random" numbers that are actually produced by a formula starting from one number, the **seed**. The same seed always gives the same "random" choices. In our project, the seed decides which 50 start–goal pairs are tested.

- Using the **same seed** again gives exactly the same test, so anyone can repeat our experiment.
- Using **different seeds** gives different test cases. If the result is similar on all seeds (as ours is), it is not just luck.

10. Results

10.1 Reproducing DINO-WM

Task	Seeds	Ours: one plan	Ours: with replanning	Published
Push-T	3	0.87	0.90	0.86 / 0.90
Wall	3	0.52	0.95	0.74 / 0.96
PointMaze	1	0.86	1.00	0.80 / 0.98

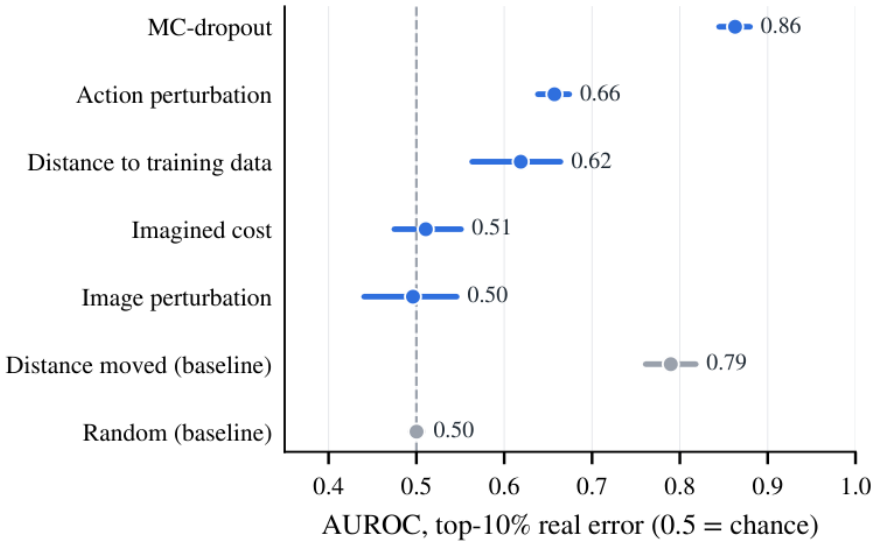


Success rate: ours (blue) and the published DINO-WM results (grey).

With replanning we match the published results on all three tasks, and slightly exceed them on PointMaze. The only weaker number is Wall with a single plan (0.52 instead of 0.74); it is the same on all three seeds, and replanning brings Wall up to the published level.

10.2 Which trust score detects wrong predictions?

Trust score	AUROC (0.5 = guessing, 1.0 = perfect)
MC-dropout	**0.86**
Action perturbation	0.66
Distance to training data	0.62
Imagined cost (planner's own confidence)	0.51
Image perturbation	0.50
Baseline: distance moved	0.79
Baseline: random	0.50



AUROC of each score; the line around each dot is its 95% confidence range.

- **MC-dropout is clearly the best**, and beats the simple baseline on every seed.

- The planner's own confidence (imagined cost) is no better than guessing: a plan that looks good to the planner is not more likely to be predicted correctly.
- Image perturbation does not work because the image encoder ignores small changes to the picture.

10.3 Removing untrusted plans

Plans removed by	Chosen plan succeeds
No removal	71%
Oracle (knows the true error)	73%
Random	70%
MC-dropout	70%
Action perturbation	75%
Distance to training data	69%
Distance moved	68%

With 80 start–goal pairs, one pair is worth about 1.3%, so all these numbers are effectively the same. Even the oracle barely helps, which tells us that the trust check has to be used throughout the search, not only at the final choice.

11. What is new in our project

1. **Trust-checked planning:** a reliability check added to the planner of a frozen world model, without retraining the model.
2. **A way to measure prediction errors:** running the planner's own plans in the simulator to get the true error of every imagined step.
3. **A trust score that works without training:** MC-dropout detects wrong predictions with AUROC 0.86, on every seed.
4. **Where the check must be applied:** removing plans only at the last moment does not help, even with perfect knowledge.
5. **An efficient reproduction:** DINO-WM on a single 8 GB laptop GPU, 3.8 times faster, with identical decisions.

12. Limitations and future work

- All three tasks are simulations, not a real robot.
- The trust scores and the gate test were run on the Push-T task.
- Planning takes minutes per decision on our GPU, so it does not run in real time.

Future work: use the trust check inside every CEM iteration on all three tasks, combine several trust scores, and test under changed visual conditions (different lighting or colours).

13. Glossary

Term	Meaning
AUROC	How often a score ranks a wrong prediction above a correct one. 0.5 = guessing, 1.0 = perfect.
Baseline	A simple method that a new method must beat to be worth anything.
Bootstrap	Repeating a calculation on random re-samples of the data to see how much the result could vary.
CEM	Cross-Entropy Method: imagine 300 plans, keep the best 30, make new ones like them, repeat.
Confidence range (95%)	The range in which the true value lies with 95% certainty.
DINO-WM	The published world model and planner we build on (ICML 2025).
DINOv2	A pretrained image network that turns a picture into a compact list of numbers.
Dropout	Randomly switching off parts of a neural network, normally used during training.
Encoder	The part that turns a picture into a compact state.
Episode	One complete attempt to reach a goal.
Frozen	Used without any changes or retraining.
Gate	The part of SCoPe that removes plans with untrusted steps.
JEPA	Joint-Embedding Predictive Architecture: a model that predicts the future in compact form, not as pixels.
Latent state	The compact list of numbers that describes a picture.
MC-dropout	Predicting several times with dropout on; disagreement means low trust.
MDP	Markov Decision Process: the formal description of states, actions, transitions and goal.
Model-based RL	RL in which the agent learns a world model and plans with it.
MPC	Model Predictive Control: plan, perform only the first part, look again, plan again.
Oracle	A reference that cheats by knowing the true answer; shows the best possible result.
Round	One plan-and-act cycle. Up to 3 per episode.
Seed	A number that fixes which test cases are picked, so experiments can be repeated exactly.
Trust region	The situations in which the world model's predictions can be trusted.
Trust score	A number for each imagined step: higher means less trustworthy.
World model	A neural network that predicts what happens after an action: the agent's imagination.